Лабораторная работа 1:
Методы градиентного спуска и метод Ньютона.

1 Алгоритмы

1.1 Методы спуска: Общая концепция

Рассматриваем задачу гладкой безусловной оптимизации:

$$\min_{x \in \mathbb{R}^n} f(x)$$

Методы спуска итеративно строят последовательность точек $(x_k)_{k=0}^{\infty}$ из $\mathbb{R}^n$ по правилу

$$x_{k+1} = x_k + \alpha_k d_k.$$

Число $k = 0, 1, \ldots$ называется номером итерации метода. Скаляр $\alpha_k \geq 0$ называется длиной шага, а вектор $d_k \in \mathbb{R}^n$ называется направлением поиска. В методах спуска требуется, чтобы направление поиска d_k являлось направлением спуска для функции f в точке x_k, т. е. удовлетворяло неравенству

$$\langle \nabla f(x_k), d_k \rangle < 0$$

В этом случае можно гарантировать, что для всех достаточно маленьких α_k значение функции f в новой точке x_{k+1} уменьшится:

$$f(x_{k+1}) < f(x_k).$$

Общая схема метода спуска приведена ниже:

Алгоритм 1 Общая схема метода спуска

Вход: Начальная точка x_0; максимальное число итераций K.

```
for k ← 0 to K do
    (Вызов оракула) Вычислить f(x_k), ∇f(x_k) и пр.
    (Критерий остановки) Если выполнен критерий остановки, то выход.
    (Вычисление направления) Вычислить направление спуска d_k.
    (Линейный поиск) Найти подходящую длину шага α_k.
    (Обновление) x_{k+1} ← x_k + α_k d_k.
end for
```

Выход: Последняя вычисленная точка x_k

1.2 Критерий остановки

Идеальным критерием остановки в методе является проверка условия $f(x_k) - f^* < \tilde{\varepsilon}$, где f^*- минимальное значение функции f, а $\tilde{\varepsilon} > 0-$ заданная точность. Такой критерий целесообразно использовать, если оптимальное значение функции f^* известно. К сожалению, зачастую это не так, и поэтому нужно использовать другой критерий. Наиболее популярным является критерий, основанный на норме градиента: $\|\nabla f(x_k)\|_2^2 < \tilde{\varepsilon}$. Квадрат здесь ставят за тем, что для «хороших» функций невязка по функции $f(x_k) - f^*$ имеет тот же порядок, что и $\|\nabla f(x_k)\|_2^2$, а не $\|\nabla f(x_k)\|_2$[1]; например, если $\|\nabla f(x_k)\|_2 \sim 10^{-5}$, то $f(x_k) - f^* \sim 10^{-10}$. Наконец, для того, чтобы критерий не зависел от того, измеряется ли функция f «в метрах»или «в километрах» (т. е. не изменялся при переходе от функции f к функции tf, где $t > 0$), то имеет смысл использовать следующий относительный вариант критерия:

$$\|\nabla f(x_k)\|_2^2 \leq \varepsilon \|\nabla f(x_0)\|_2^2 \tag{1}$$

где $\varepsilon \in (0, 1)$ - заданная относительная точность. Таким образом, критерий остановки (1) гарантирует, что метод уменьшит начальную невязку $\|\nabla f(x_0)\|_2$ в ε^{-1} раз. В этом задании Вам нужно будет во всех методах использовать критерий остановки (1) .

1.3 Линейный поиск

Рассматривается функция

$$\phi_k(\alpha) := f(x_k + \alpha d_k)$$

Заметим, что

$$\phi_k'(\alpha) = \langle \nabla f(x_k + \alpha d_k), d_k \rangle$$

Поскольку d_k является направлением спуска, то $\phi'(0) = \langle \nabla f(x_k), d_k \rangle < 0$.

Условием Армихо для α называется выполнение следующего неравенства:

$$\phi_k(\alpha) \leq \phi_k(0) + c_1 \alpha \phi_k'(0)$$

где $c_1 \in (0, 0.5)$ - некоторая константа.

Для поиска точки α, удовлетворяющей условию Армихо, обычно используют следующую процедуру - метод дробления шага (бэктрекинг):

Алгоритм 2 Метод дробления шага

Вход: Функция $\phi_k : \mathbb{R}_+ \to \mathbb{R}$. Начальная точка: $\alpha_k^{(0)}$.

```
α ← α_k^(0).
while φ_k(α) > φ(0) + cαφ'_k(0) do
    α ← α/2.
end while
```

Выход: α

«Адаптивный» метод подбора шага запоминает величину α_k, найденную на текущей итерации и на следующей итерации начинает процедуру дробления с $\alpha_{k+1}^{(0)} := 2\alpha_k$. Исключение здесь составляют ньютоновские и

[1] Например, это верно для сильно-выпуклых функций с липшицевым градиентом

квазиньютоновские методы - в этих методах процедуру дробления шага всегда нужно начинать с $\alpha_k^{(0)} := 1$

Сильные условия Вульфа:

$$\phi_k(\alpha) \leq \phi(0) + c_1 \alpha \phi_k'(0)$$
$$|\phi_k'(\alpha)| \leq c_2 \, |\phi_k'(0)|$$

Здесь $c_1 \in (0, 0.5), c_2 \in (c_1, 1)$.

Самостоятельно реализовывать схему для сильных условий Вульфа не нужно. Используйте библиотечную реализацию (функция scalar_search_wolfe2 из модуля scipy.optimize.linesearch). В ней начальная длина шага $\alpha_k^{(0)}$ автоматически выбирается равной 1.

1.4 Градиентный спуск

Градиентный спуск:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

Можно рассматривать как метод спуска, в котором направление поиска d_k равно антиградиенту $-\nabla f(x_k)$. Длина шага α_k выбирается с помощью линейного поиска.

1.5 Метод Ньютона

Метод Ньютона:

$$x_{k+1} = x_k - \alpha_k \left[\nabla^2 f(x_k)\right]^{-1} \nabla f(x_k).$$

Для метода Ньютона очень важно использовать единичный шаг $\alpha_k = 1$, чтобы обеспечить локальную квадратичную сходимость. Поэтому в алгоритмах линейного поиска нужно всегда первым делом пробовать единичный шаг. Теория гарантирует, что в зоне квадратичной сходимости метода Ньютона единичный шаг будет удовлетворять условиям Армихо/Вульфа, и поэтому автоматически будет приниматься. Если единичный шаг не удовлетворяет условиям Армихо/Вульфа, то алгоритмы линейного поиска его уменьшат и, тем самым, обеспечат глобальную сходимость метода Ньютона.

Вычисление Ньютоновского направления $d_k = -\left[\nabla^2 f(x_k)\right]^{-1} \nabla f(x_k)$ эквивалентно решению линейной системы уравнений:

$$\nabla^2 f(x_k) \, d_k = -\nabla f(x_k)$$

Если гессиан - положительно определённая матрица: $\nabla^2 f(x_k) \succ 0$, то предпочтительным методом решения такой системы является разложение Холецкого, которое также, как и метод Гаусса, работает за $O\left(n^3\right)$, но является вычислительно более эффективным. Если матрица системы не является положительно определённой, то метод Холецкого сможет обнаружить и сообщить об этом.

2 Модели

В рамках лабораторной работы предлагается проанализировать работу метода Ньютона и градиентного спуска на следующих моделях:

- Выпуклая квадратичная функция;
- Невыпуклая тестовая функция;
- ML-оракулы для задач регрессии и классификации.

2.1 Задачи машинного обучения: Минимизация эмпирического риска

С математической точки зрения, обучение линейных моделей машинного обучения сводится к задаче безусловной оптимизации. Неформально задача формулируется следующим образом. Имеется обучающая выборка $((a_i, b_i))_{i=1}^m$, состоящая из m пар. Вектор $a_i \in \mathbb{R}^n$, содержит известные характеристики объекта (называемые признаками). Величина b_i - это правильный ответ (целевая переменная), который мы хотим научиться предсказывать для новых объектов.

Мы рассматриваем семейство линейных моделей, где предсказание строится на основе скалярного произведения вектора признаков a_i и вектора неизвестных параметров $x \in \mathbb{R}^n$ (весов модели).

Чтобы найти оптимальный вектор параметров x, необходимо минимизировать расхождение между предсказаниями модели $< a_i, x >$ и истинными ответами b_i на всей обучающей выборке. Для измерения этого расхождения вводится гладкая функция потерь **(loss function)** L. Общая оптимизационная задача, называемая задачей минимизации эмпирического риска с L_2-регуляризацией, имеет вид:

$$f(x) = \frac{1}{m}\sum_{i=1}^{m} L(b_i, < a_i, x >) + \frac{\lambda}{2}||x||_2^2 \to \min_{x \in \mathbb{R}^n} \tag{2}$$

Слагаемое $\frac{\lambda}{2}||x||_2^2$ (где $\lambda > 0$) называется регуляризатором. В контексте машинного обучения оно предотвращает переобучение, не позволяя компонентам вектора x становиться аномально большими.

Однако для нас гораздо важнее вычислительная роль регуляризации. С точки зрения методов оптимизации, добавление $\frac{\lambda}{2}||x||_2^2$ к функции $f(x)$ означает, что к гессиану целевой функции прибавляется диагональная матрица λI:

$$\nabla^2 f(x) = \nabla^2 \left(\frac{1}{m}\sum_{i=1}^{m} L \right) + \lambda I \tag{3}$$

Это искусственно смещает спектр матрицы Гессе вправо на величину λ, гарантируя, что гессиан будет строго положительно определенным $\nabla^2 f(x) \succ 0$. Это обеспечивает глобальную сильную выпуклость задачи и спасает метод Ньютона от ошибок при разложении Холецкого, особенно в ситуациях, когда признаков больше, чем объектов ($n > m$), или когда признаки линейно зависимы.

В зависимости от природы целевой переменной b_i, задачи делятся на два основных класса:

2.1.1 Задача регрессии (Regression)

В задачах регрессии требуется предсказывать непрерывную величину: $b_i \in \mathbb{R}$ (например, температуру, стоимость квартиры или уровень спроса). В этом случае предсказанием модели является само значение скалярного произведения $< a_i, x >$, а функция потерь штрафует модель за алгебраическую разность (невязку) между ответом и предсказанием.

2.1.2 Задача бинарной классификации (Binary Classification)

В задачах классификации требуется распределить объекты по двум непересекающимся категориям, поэтому ответы дискретны: $b_i \in -1, 1$ (например, спам / не спам, болен / здоров). Предсказание класса осуществляется по знаку скалярного произведения: $class(a_i) = \text{sign}(< a_i, x >)$. При этом абсолютная величина $| < a_i, x > |$ интерпретируется как «уверенность» модели в своем предсказании.

Для удобства оптимизации в классификации часто вводят понятие отступа (margin): $M_i = b_i < a_i, x >$

Если $M_i > 0$, знак предсказания совпал с истинным классом b_i (объект классифицирован верно). Если $M_i < 0$, модель допустила ошибку.

Функции потерь для классификации обычно строятся как строго убывающие функции от отступа $L(M_i)$, которые сильно штрафуют отрицательные отступы и дают околонулевой штраф за большие положительные отступы.

Важно: Конкретный вид функций потерь L, с которыми предстоит работать вашей команде, а также наборы реальных данных для экспериментов, определяются индивидуальным вариантом (см. Приложение).

Замечание (Почему модель линейная?): В данной лабораторной работе мы ограничиваемся линейными моделями, где параметры x входят в предсказание только в виде скалярного произведения $< a_i, x >$. С точки зрения численных методов оптимизации это делается ради одного критически важного свойства – сохранения выпуклости. Композиция строго выпуклой функции потерь L и линейной функции $< a_i, x >$ гарантирует, что итоговая целевая поверхность $f(x)$ не имеет локальных минимумов, отличных от глобального. Кроме того, линейность модели позволяет получить аналитически красивую структуру матрицы Гессе в виде $\nabla^2 f(x) = \frac{1}{m} A^T D A + \lambda I$, что делает возможным точный анализ ее спектра и числа обусловленности, а также эффективное применение метода Ньютона. В нелинейных моделях (например, глубоких нейронных сетях) гессиан, как правило, не является положительно определенным, из-за чего классический метод Ньютона неприменим без сложных аппроксимаций.

2.2 Разностная проверка градиента и гессиана

Проверить правильность реализации подсчета градиента можно с помощью конечных разностей:

$$[\nabla f(x)]_i \approx \frac{f(x + \varepsilon_1 e_i) - f(x)}{\varepsilon_1}$$

где $e_i := (0, \ldots, 0, 1, 0, \ldots, 0)$ – i-й базисный орт, а ε_1– достаточно маленькое положительное число: $\varepsilon_1 \sim \sqrt{\varepsilon_{\text{mach}}}$, где $\varepsilon_{\text{mach}}$ – машинная точность

$\left(\approx 10^{-16}\right.$ для типа double).

Вторые производные:

$$\left[\nabla^2 f(x)\right]_{ij} \approx \frac{f\left(x+\varepsilon_2 e_i+\varepsilon_2 e_j\right)-f\left(x+\varepsilon_2 e_i\right)-f\left(x+\varepsilon_2 e_j\right)+f(x)}{\varepsilon_2^2}$$

Здесь $\varepsilon_2 \sim \sqrt[3]{\varepsilon_{\text{mach}}}$.

3 Формулировка задания

Лабораторная работа выполняется в командах по 3 человека (можно меньше). Каждая команда получает индивидуальный вариант (см. Приложение A). Вариант строго определяет:

- какую невыпуклую 2D-функцию нужно использовать в п. 3.2;
- какие две модели машинного обучения (и наборы данных к ним) нужно реализовать;
- какой углубленный исследовательский трек нужно выполнить дополнительно.

В отчете необходимо явно указать, как были распределены задачи между участниками команды.

3.1 План выполнения

1. Даны прототипы функции, которые Вам нужно будет реализовать. Некоторые процедуры уже частично или полностью реализованы.

2. Реализовать метод градиентного спуска (функция gradient_descent в модуле optimization) и процедуру линейного поиска (метод line_search в классе LineSearchTool в модуле optimization).

Рекомендация: Для поиска точки, удовлетворяющей сильным условиям Вульфа, воспользуйтесь библиотечной функцией scalar_search_wolfe2 из модуля scipy.optimize.linesearch. Однако следует иметь в виду, что у этой библиотечной функции имеется один недостаток: она иногда не сходится и возвращает значение None. Если библиотечный метод вернул None, то запустите процедуру дробления шага (бэктрекинг) для поиска точки, удовлетворяющей условию Армихо.

3. Получить формулы для градиента и гессиана функций потерь для двух функций потерь (моделей), указанных в вашем варианте. Обязательно учтите наличие слагаемого L_2-регуляризации (регуляризация Тихонова). Выписать их в отчет в матрично-векторной форме[2] с использованием поэлементных функций, но без каких-либо суммирований. Также выписать в

[2] В матрично-векторной форме допускается использование операций матричного сложения/произведения, умножения на скаляр, транспонирования, стандартного скалярного произведения, поэлементного произведения (произведение Адамара; символ $\odot$), а также применения ко всем элементам вектора некоторой скалярной функции. Кроме этого, допускается использование стандартных матриц/векторов (заданного размера): единичная матрица I_n, нулевая матрица $0_m \times$, нулевой вектор 0_n, вектор из всех единиц $1_n := (1, \ldots, 1)$.

отчет выражение для самой функции логистической регрессии в матрично-векторной форме (без явных суммирований).

4. Реализовать классы-оракулы для ваших моделей в модуле в модуле oracles).

Замечание: Реализация оракула должна быть полностью векторизованной, т. е. код не должен содержать никаких циклов for.

Замечание: Ваш код должен поддерживать как плотные матрицы A типа np.array, так и разреженные типа scipy.sparse.csr_matrix.

Замечание: При реализации функций потерь, содержащих экспоненты или логарифмы (например, Logistic, Poisson, Exponential, Log-Cosh, Probit), крайне велик риск вычислительного переполнения (OverflowError, inf, NaN) или взятия логарифма от нуля. Вы обязаны использовать математически стабильные функции из библиотек numpy и scipy.special (например, np.logaddexp, scipy.special.log_ndtr и т.д.) или применять аккуратное клиппирование (np.clip), чтобы код работал стабильно при любых (даже плохих) значениях весов.

5. Реализовать подсчет разностных производных (функции grad_finite_diff и hess_finite_diff в модуле oracles). Проверить правильность реализации подсчета градиента и гессиана логистического оракула с помощью реализованных функций. Для этого сгенерируйте небольшую модельную выборку (матрицу A и вектор b) и сравните значения, выдаваемые методами grad и hess, с соответствующими разностными аппроксимациями в нескольких пробных точках x.

6. Реализовать метод Ньютона (функция newton в модуле optimization).

Замечание: Для поиска направления в методе Ньютона не нужно в явном виде обращать гессиан (с помощью функции np.linalg.inv) или использовать самый общий метод для решения системы линейных уравнений (numpy.linalg.solve). Вместо этого следует учесть тот факт, что в рассматриваемой задаче (благодаря регуляризации) гессиан является симметричной положительно определенной матрицей и воспользоваться разложением Холецкого (функции scipy.linalg.cho_factor и scipy.linalg.cho_solve).

7. Провести базовые эксперименты (описанные в разделе 3.1–3.6), строго используя 2D-функцию, Оракулы и датасеты из вашего варианта. Написать отчет.

8. **Углубленный исследовательский трек:** Выполнить задание из вашего индивидуального трека (см. описание треков в Приложении). Результаты этого исследования должны быть оформлены как отдельный, финальный раздел вашего отчета (минимум 1 страница аналитики с графиками и математическими выводами).

3.2 Эксперимент: Траектория градиентного спуска на квадратичной функции

В этом эксперименте вам предстоит визуально проанализировать поведение Градиентного спуска (ГС) и метода Ньютона на двумерных поверхностях разной степени сложности. Эксперимент состоит из двух частей.

3.2.1 Часть 1. Квадратичные функции (Анализ обусловленности)

Рассматривается функция $f(x) = (1/2)\langle Ax, x\rangle - \langle b, x\rangle$, где $A \in \mathbb{S}^n_{++}, b \in \mathbb{R}^n$. Оракул для нее QuadraticOracle уже реализован в модуле oracles.

1. Придумайте и задайте две квадратичные двумерные функции:

а) Хорошо обусловленную (линии уровня близки к окружностям, число обусловленности $\kappa \approx 1$).

б) Плохо обусловленную (линии уровня — сильно вытянутые эллипсы, $\kappa \gg 1$)

2. Запустите ГС и метод Ньютона из одной и той же начальной точки x_0.

3. Постройте графики с линиями уровня (контурами) этих функций и наложите поверх них траектории шагов обоих методов.

3.2.2 Часть 2. Невыпуклая функция (Анализ сложной геометрии)

Квадратичные формы имеют лишь один глобальный минимум. Теперь исследуйте поведение методов на сложной невыпуклой 2D-функции, указанной в вашем варианте (Розенброка, Химмельблау или Била). Особый интерес здесь представляет то, к какой критической точке $\nabla f = 0$ сойдется метод в зависимости от начального приближения.

1. Реализуйте оракул для выданной вам функции (выведите аналитически ∇f и $\nabla^2 f$.

2. Постройте линии уровня этой функции на достаточно широком интервале, чтобы были видны е ключевые особенности (овраги, седловые точки или несколько минимумов).

3. Выберите 3–4 различные начальные точки x_0 (например, на дне оврага, на крутом склоне, рядом с локальным максимумом/седлом) и постройте траектории ГС и метода Ньютона.

4. Карта бассейнов притяжения. Задайте на плоскости сетку начальных точек (например, 200×200 в квадрате $[-5, 5] \times [-5, 5]$. Запустите алгоритм из каждой точки сетки. Покрасьте каждую точку сетки в определенный цвет в зависимости от того, в какой именно минимум сошелся алгоритм (например, для функции Химмельблау будет 4 цвета для 4-х минимумов). Если алгоритм не сошелся за отведенное число итераций или ушел в бесконечность – покрасьте точку в черный цвет. Если алгоритм сошелся не к минимуму, а к седловой точке или локальному максимуму – используйте отдельный специфический цвет (например, серый).

3.2.3 Вопросы для отчета

1. Как отличается поведение ГС в зависимости от числа обусловленности κ и стратегии выбора шага (константная, Армихо, Вульф)?

2. Как ведет себя метод Ньютона на квадратичной функции по сравнению с ГС? За сколько итераций он сходится и почему?

3. Сошлись ли методы в один и тот же минимум на вашей невыпуклой функции из разных стартовых точек? Сталкивались ли вы с ошибкой LinAlgError (матрица не положительно определена) при запуске метода Ньютона на невыпуклой функции? Если да, объясните с точки зрения

спектра матрицы Гессе, почему в этой точке пространства метод Ньютона математически неприменим в своем классическом виде.

3.1. Почему ГС почти никогда не сходится к седловым точкам, даже если стартует недалеко от них?

3.2. Почему «чистый» метод Ньютона (без модификаций гессиана) может сходиться к седловым точкам или даже локальным максимумам? Как выглядят границы между бассейнами притяжения у метода Ньютона? Наблюдаете ли вы «фрактальную» структуру на границах (фракталы Ньютона)? **Модификации:** Если ваша команда выполняет Трек 1 (модификация гессиана $H+\gamma I$), постройте карту притяжения для модифицированного метода Ньютона. Исчезли ли зоны сходимости к седловым точкам? Стала ли карта более «упорядоченной» и похожей на карту ГС?

Для рисования линий уровня можете воспользоваться функцией plot_levels, а для рисования траекторий - plot_trajectory из файла plot_trajectory_2d.py, прилагающегося к заданию. Для отрисовки карт притяжения удобно использовать plt.imshow() или plt.pcolormesh().

3.3 Эксперимент: Зависимость числа итераций градиентного спуска от числа обусловленности и размерности пространства

Исследуйте, как зависит число итераций, необходимое градиентному спуску для сходимости, от следующих двух параметров: 1) числа обусловленности $\kappa \geq 1$ оптимизируемой функции и 2) размерности пространства n оптимизируемых переменных. Для этого эксперимента мы возвращаемся к квадратичной задаче $f(x) = \frac{1}{2}\langle Ax, x\rangle - < b, x >$

3.3.1 Методика генерации данных

Для этого для заданных параметров n и κ сгенерируйте случайным образом квадратичную задачу размера n с числом обусловленности κ и запустите на ней градиентный спуск с некоторой фиксированной требуемой точностью. Замерьте число итераций $T(n, \kappa)$, которое потребовалось сделать методу до сходимости (успешному выходу по критерию остановки).

Рекомендация: Проще всего сгенерировать случайную квадратичную задачу размера n с заданным числом обусловленности κ следующим образом. В качестве матрицы $A \in \mathbb{S}^n_{++}$ удобно взять просто диагональную матрицу $A = \mathrm{Diag}(a)$, у которой диагональные элементы сгенерированы случайно в пределах $[1, \kappa]$, причем $\min(a) = 1, \max(a) = \kappa$. В качестве вектора $b \in \mathbb{R}^n$ можно взять вектор со случайными элементами. Диагональные матрицы удобно рассматривать, поскольку с ними можно эффективно работать даже при больших значениях n. Рекомендуется хранить матрицу A в формате разреженной диагональной матрицы (см. scipy.sparse.diags).

3.3.2 Порядок проведения эксперимента

1. Зафиксируйте некоторое значение размерности n.

2. Переберите различные числа обусловленности κ по логарифмической сетке и постройте график зависимости $T(\kappa, n)$ от κ. Поскольку каждый раз квадратичная задача генерируется случайным образом, то повторите этот

эксперимент несколько раз. В результате для фиксированного значения n у Вас должно получиться целое семейство кривых зависимости $T(\kappa, n)$ от κ. Нарисуйте все эти кривые одним и тем же цветом для наглядности (например, красным).

3. Теперь увеличьте значение n и повторите эксперимент снова. Вы должны получить новое семейство кривых $T(n', \kappa)$ против κ. Нарисуйте их все одним и тем же цветом, но отличным от предыдущего (например, синим).

4. Повторите эту процедуру несколько раз для других значений n. В итоге должно получиться несколько разных семейств кривых - часть красных (соответствующих одному значению n), часть синих (соответствующих другому значению n), часть зеленых и т. д.

5. Обратите внимание, что значения размерности n имеет смысл перебирать по логарифмической сетке (например, $n = 10, n = 100, n = 1000$ и т. д.).

3.3.3 Вопросы для отчета

1. Какие выводы можно сделать из полученной картинки?

2. Зависит ли (и если да, то как) число итераций ГС от размерности пространства n при фиксированном числе обусловленности κ ?

3. Что произошло с Методом Ньютона при росте κ? Объясните полученный результат (число итераций Ньютона), опираясь на математическую формулу шага метода Ньютона для квадратичной функции.

3.4 Эксперимент: Сравнение методов градиентного спуска и Ньютона на реальных задачах машинного обучения

В этом эксперименте необходимо сравнить эффективность Градиентного спуска и метода Ньютона на задачах обучения моделей из вашего варианта (одна задача регрессии и одна задача классификации).

3.4.1 Порядок проведения

1. Используйте наборы данных (датасеты) с сайта LIBSVM[3] , закрепленные за вашими моделями в Приложении А.

2. Для каждой модели (оракула) установите стандартный коэффициент регуляризации: $\lambda = 1/m$.

3. Параметры обоих методов взять равными параметрам по умолчанию. Начальную точку выбрать $x_0 = 0$.

4. Построить графики сходимости следующих двух видов:

(a) Зависимость значения функции от реального времени работы метода (в секундах).

(b) Зависимость относительного квадрата нормы градиента $\|\nabla f(x_k)\|_2^2 / \|\nabla f(x_0)\|_2^2$ (в логарифмической шкале) против реального времени работы.

При этом оба метода (градиентный спуск и Ньютон) нужно рисовать на одном и том же графике.

[3] http://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/

Укажите в отчете, какова стоимость итерации и сколько памяти требуется каждому из методов в зависимости от параметров m (размер выборки) и n (размерность пространства). При оценке используйте нотацию $O(\cdot)$, скрывающую внутри себя абсолютные константы.

Какие выводы можно сделать по результатам этого эксперимента? Какой из методов лучше и в каких ситуациях?

Рекомендация: Любой набор данных с сайта LIBSVM представляет из себя текстовый файл в формате svmlight. Чтобы считать такой текстовый файл, можно использовать функцию load_svmlight_file из модуля sklearn.datasets. Обратите внимание, что эта функция возвращает матрицу в формате scipy.sparse.csr_matrix, поэтому Ваша реализация оракула должна поддерживать такие матрицы.

3.5 Эксперимент: Стратегия выбора длины шага в градиентном спуске

Исследовать, как зависит поведение метода от стратегии подбора шага: константный шаг (попробовать различные значения), бэктрэкинг (попробовать различные константы c), условия Вульфа (попробовать различные параметры c_2).

Рассмотрите квадратичную функцию и ML модели вашего варианта.

Запустите для этих функций градиентный спуск с разными стратегиями выбора шага из одной и той же начальной точки.

Нарисуйте кривые сходимости (относительная невязка по функции в логарифмической шкале против числа итераций - для квадратичной функции, относительный квадрат нормы градиента в логарифмической шкале против числа итераций - для ML оракула) для разных стратегий на одном графике.

Попробуйте разные начальные точки. Ответьте на вопрос: Какая стратегия выбора шага является самой лучшей?

3.6 Эксперимент: Стратегия выбора длины шага в методе Ньютона

Повторите эксперимент 3.5 со сравнением стратегий выбора шага на ML моделях вашего варианта, но для метода Ньютона. Какая стратегия работает лучше всего?

Почему для метода Ньютона единичный шаг является приоритетным?

Зафиксируйте момент, когда алгоритм входит в зону квадратичной сходимости (шаг $\alpha = 1$ начинает приниматься всегда). Как меняется значение ∇f на этом этапе?

3.7 Эксперимент: Влияние регуляризации на обусловленность и скорость сходимости

Коэффициент L_2-регуляризации λ напрямую контролирует константу сильной выпуклости вашей целевой функции. Чем больше λ, тем лучше обусловлена задача оптимизации, но тем дальше ее минимум от минимума исходной (нерегуляризованной) функции.

3.7.1 Порядок проведения

1. Проведите эксперимент для обеих моделей вашего варианта. Сравните, для какой из функций потерь (регрессионной или классификационной) изменение λ сильнее влияет на устойчивость метода Ньютона.

2. Выберите из вашего варианта датасеты, где число признаков n наибольшее. Зафиксируйте начальную точку x_0.

3. Запустите Градиентный спуск и метод Ньютона для значений λ по логарифмической сетке: $\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 0.1, 1.0, 10.0\}$.

4. Постройте графики:

а) Зависимость количества итераций, потребовавшихся для сходимости (отдельно кривая для GD, отдельно для Ньютона) от λ

б) Зависимость итогового времени работы до сходимости от λ

4. Проверьте, как меняется норма весов $||x^*||_2$ в найденной точке минимума при росте λ.

3.7.2 Вопросы для отчета

1. Как изменяется количество итераций рассматриваемых методов при стремлении λ к нулю и почему?

2. Возникала ли численная неустойчивость методов при некоторых значениях λ, если да, то почему и как вы ее решили?

3. Как λ влияет на «зону квадратичной сходимости» метода Ньютона? Становится ли она шире при увеличении регуляризации?

4. Наблюдаете ли вы плато в количестве итераций при больших λ? Почему?

3.8 Углубленное исследование: Индивидуальный трек команды

В данном разделе команда должна представить результаты исследования по своему индивидуальному треку (см. Приложение А). Это задание требует не просто запуска готовых функций, а постановки отдельного вычислительного эксперимента.

Отчет по треку должен содержать:

1. Постановку задачи: Что именно исследовалось.

2. Описание эксперимента: Какие параметры варьировались, на каких данных проводились тесты.

3. Результаты: Графики, таблицы, профили времени работы.

4. Выводы: Глубокая аналитика полученных результатов. Почему метод вел себя именно так? Подтверждают ли результаты теорию из лекций?

4 Оформление задания

Работа команды сдается в виде трех взаимосвязанных артефактов:

1. Репозиторий на Github/Gitlab Каждая команда создает один общий приватный репозиторий (с добавлением проверяющих преподавателей). Репозиторий должен иметь следующую структуру:

Все результаты должны быть воспроизводимыми. Если вы используете случайность - зафиксируйте seed.

```
opt_project_team_N/
├── README.md              # Название трека, состав команды, инструкция по запуску
├── requirements.txt       # Фиксация версий библиотек (scipy, numpy, scikit-learn и др.)
├── src/                   # Исходный код (Никаких вычислений, только классы и функции)
│   ├── optimization.py    # Реализация GD, Newton, Line Search
│   └── oracles.py         # Реализация функций, градиентов, Гессианов
├── notebooks/             # Jupyter Notebooks для экспериментов
│   ├── base_checks.ipynb  # Проверка базовой части (сходимость, разностный градиент)
│   └── research_track.ipynb # Код вашего исследовательского трека
└── report/                # Материалы отчета
    ├── final_report.pdf   # Итоговый отчет
    └── presentation.pdf   # Слайды для защиты
```

Рис. 1: Структура репозитория

2. Отчет в PDF (рекомендуется использовать LaTeX)

Каждый проведенный эксперимент следует оформить как отдельный раздел в PDF документе (название раздела - название соответствующего эксперимента). Для каждого эксперимента необходимо сначала написать его описание:

а) постановка задачи

б) какие функции оптимизируются, каким образом генерируются данные, Описание датасетов (размерность $m \times n$, плотность/разреженность матриц), используемого "железа"(CPU/RAM), какие методы и с какими параметрами используются.

в) результаты Далее должны быть представлены результаты соответствующего эксперимента - графики, таблицы и т. д.

г) наконец, после результатов эксперимента должны быть написаны Ваши выводы - какая зависимость наблюдается и почему. Как это бьется с теорией оптимизации из лекций?

Важно: Отчет не должен содержать никакого кода. Каждый график должен быть прокомментирован - что на нем изображено, какие выводы можно сделать из этого эксперимента. Обязательно должны быть подписаны оси. Если на графике нарисовано несколько кривых, то должна быть легенда. Сами линии следует рисовать достаточно толстыми, чтобы они были хорошо видимыми.

3. Презентация для защиты

5 Приложение А. Пакеты Оракулов и Варианты заданий

5.1 Невыпуклые тестовые функции

1. Функция Розенброка

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2$$

2. Функция Химмельблау

$$f(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2$$

3. Функция Била

$$f(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2$$

5.2 Функции потерь для ML задач

Каждая команда реализует один Пакет Оракулов, состоящий из одной задачи регрессии и одной задачи бинарной классификации.

Для каждой модели необходимо провести эксперименты на двух наборах данных, обладающих принципиально разными свойствами:

- «Высокий» датасет ($m \gg n$): Количество объектов многократно превышает количество признаков.
- «Широкий» или Разреженный датасет ($n \gg m$ или n велико): Количество признаков огромно.

Во всех формулах ниже $z = \langle a_i, x \rangle$ - предсказание линейной модели для объекта a_i. Не забудьте, что полная целевая функция включает L_2-регуляризатор:$f(x) = \frac{1}{m} \sum L(y_i, z_i) + \frac{\lambda}{2} ||x||_2^2$.

5.2.1 Пакет 1

Регрессия Log-Cosh Loss

1. Функция $L(y, z) = \ln(\cosh(z - y))$, $y \in \mathbb{R}$
2. Свойство (оптимизационное): Строго выпуклая, бесконечно дифференцируемая функция. Около нуля ведет себя как парабола, но при росте невязки $|z-y|$ вторая производная экспоненциально быстро убывает к нулю. Вдали от оптимума матрица Гессе становится вычислительно вырожденной, из-за чего шаг метода Ньютона может стать неадекватно огромным.
3. Практический смысл (ML): Применяется для робастной регрессии. В отличие от обычной квадратичной ошибки (MSE), которая квадратично штрафует аномалии и сильно искажает веса модели, Log-Cosh штрафует выбросы линейно (как модуль ошибки, MAE), не позволяя им сломать предсказания модели для нормальных данных.
4. Датасеты: *abalone* и *bodyfat*

Классификация Exponential Loss (AdaBoost)

1. Функция $L(y, z) = \exp(-yz)$, $y \in \{-1, 1\}$
2. Свойство (оптимизационное): Экспоненциально штрафует неверные ответы. Крайне чувствительна к выбросам: градиент и гессиан могут переполнить тип float64 без жесткого линейного поиска.
3. Практический смысл (ML): Эта функция лежит в основе классического алгоритма AdaBoost. Она невероятно агрессивно штрафует модель за неверно классифицированные объекты, заставляя алгоритм фокусироваться на самых «сложных» примерах выборки. Очень чувствительна к неверно размеченным данным.
4. Датасеты: *a9a* и *gisette*

5.2.2 Пакет 2

Регрессия Poisson Regression Loss

1. Функция $L(y, z) = \exp(z) - yz$, $y >= 0$
2. Свойство (оптимизационное): Строго выпуклая, но сильно асимметричная функция. Гессиан растет экспоненциально при $z > 0$, требуя идеальной настройки условий Вульфа/Армихо и инициализации весов нулями ($x_0 = 0$)
3. Практический смысл (ML): Классически применяется для предсказания счетчиков (числа событий), но в машинном обучении этот Loss часто используют для любых строго положительных непрерывных величин с "тяжелым правым хвостом"(например, цен на недвижимость в датасете cadata). Это называется методом квази-максимального правдоподобия (Quasi-Poisson). Модель $E[y] = \exp(z)$ гарантирует, что предсказанная цена никогда не станет отрицательной.
4. Требование к данным: Если вы используете отмасштабированные данные, перед оптимизацией обязательно сдвиньте вектор ответов $y_{\text{new}} = y - \min(y)$, чтобы гарантировать $y_{\text{new}} > 0$.
5. Датасеты: *cadata* и *space_ ga*

Классификация Logistic Regression

1. Функция $L(y, z) = \ln(1 + \exp(-yz))$, $y \in \{-1, 1\}$
2. Свойство (оптимизационное): Вычислительно стабильна только при правильной реализации через специальную функцию numpy.logaddexp(0, -yz), иначе логарифмирование нуля выдаст ошибку. При линейной разделимости выборки (отступы $yz \to \infty$ гессиан обнуляется, и система уравнений Ньютона выживает только за счет регуляризатора λI.
3. Практический смысл (ML): В отличие от многих других функций, позволяет предсказывать не просто класс, а откалиброванную вероятность принадлежности к классу (через сигмоиду). Является следствием метода максимального правдоподобия для распределения Бернулли.
4. Датасеты: *phishing* и *real-sim*

5.2.3 Пакет 3

Регрессия Pseudo-Huber Loss

1. Функция $L(y, z) = \sqrt{1 + \left(\frac{z-y}{\delta}\right)^2}$, $y \in \mathbb{R}$
2. Свойство (оптимизационное): Строго выпуклая гладкая функция. Около нуля (при малых невязках) ведет себя как квадратичная функция (MSE), а на бесконечности (при больших невязках) - как линейная (MAE). Из-за линейности на "хвостах"вторая производная полиномиально убывает к нулю. Вдали от оптимума Метод Ньютона может

демонстрировать огромные хаотичные скачки или падать из-за сингулярности гессиана, если не применять регуляризацию.

3. Практический смысл (ML): Применяется для робастной регрессии. Если в обучающей выборке есть сильные аномалии (выбросы в целевой переменной y), обычная квадратичная ошибка (Ridge) сильно исказит модель, пытаясь под них подстроиться. Pseudo-Huber штрафует за выбросы линейно, ограничивая их влияние на градиент, но при этом сохраняет математическую гладкость, необходимую для методов оптимизации второго порядка.

4. Датасеты: *mg* и *triazines*

Классификация Smoothed SVM (Soft-Margin Loss)

1. Функция $L(y, z) = \ln(1 + \exp(1 - yz))$, $y \in \{-1, 1\}$

2. Свойство (оптимизационное): Смещенная версия логистической функции. Если объект классифицируется надежно $yz > 1$, слагаемое $\exp(1-yz)$ быстро достигает машинный ноль формата float64. Матрица Гессе будет состоять из нулей.

3. Практический смысл (ML): Гладкий аналог Hinge Loss, лежащего в основе Метод Опорных Векторов (SVM). Модель не просто пытается угадать правильный знак ($yz > 0$), она требует создания «зазора уверенности» (margin) не меньше единицы. Всё, что имеет зазор >1, модель перестает штрафовать.

4. Датасеты: *ijcnn1* и *rcv1.binary*

5.2.4 Пакет 4

Регрессия Ridge Regression (MSE)

1. Функция $L(y, z) = \frac{1}{2}(z - y)^2$, $y \in \mathbb{R}$

2. Свойство (оптимизационное): Идеальная квадратичная форма. Матрица Гессе является константной и не зависит от текущей точки x. Метод Ньютона (с шагом $\alpha = 1$) должен сходиться к глобальному оптимуму за 1 итерацию. (Покажите это теоретически в отчете).

3. Практический смысл (ML): Обычный метод наименьших квадратов. Оптимален в предположении, что ошибки предсказания (шум в данных) имеют нормальное (гауссовское) распределение. Сильно подвержен негативному влиянию аномальных выбросов.

4. Датасеты: *housing_scale* и *pyrim*

Классификация Probit Regression

1. Функция $L(y, z) = -\ln(\Phi(yz))$, где Φ - интегральная функция стандартного нормального распределения, $y \in -1, 1$

2. Свойство (оптимизационное): Строго выпуклая. Обычное вычисление $\Phi(x)$ при сильных отрицательных отступах (например, $x = -10$) вернет точный 0.0, после чего $\ln(0)$ обрушит алгоритм. Для стабильных вычислений на хвостах обязательно требуется использование асимптотического разложения через scipy.special.log_ndtr(yz).

3. Практический смысл (ML): Популярная в эконометрике альтернатива логистической регрессии. Строится на предположении, что скрытая (латентная) переменная, определяющая класс объекта, распределена нормально. Хвосты у нормального распределения тоньше, поэтому Probit чуть жестче штрафует сильные ошибки.

4. Датасеты: *w8a* и *colon-cancer*

5.3 Дополнительные исследовательские треки

5.3.1 Трек 1. Эвристики модификации Гессиана

На невыпуклых функциях (или вдали от оптимума на сложных функциях потерь) матрица Гессе может иметь отрицательные или нулевые собственные значения. Классический метод Ньютона в таких точках математически неприменим, а найденное направление на шаге оптимизации может даже не быть направлением убывания. Чтобы алгоритм работал, матрицу H принудительно заменяют на близкую к ней положительно определенную матрицу $\bar{H} \geqq 0$.

Задание: Выберите вашу невыпуклую 2D-функцию и ML-модель из варианта с отключенной регуляризацией ($\lambda = 0$). Реализуйте и сравните две принципиально разные стратегии модификации гессиана:

1. Динамический сдвиг спектра (Левенберг-Маркдвардт): $\bar{H} = H + \gamma I$. Перехватывайте ошибку Холецкого: если матрица не факторизуется или шаг не дает убывания функции, отменяйте шаг и увеличивайте γ в 10 раз (начиная с 10^{-5}). При успешном шаге уменьшайте γ.

2. Спектральное усечение: На каждой итерации полностью вычисляйте спектр $H = V\Lambda V^T$ (через scipy.linalg.eigh). Заменяйте все собственные значения $\lambda_i < \varepsilon$ на малое положительное число ε (например 10^{-5}) или берите их по модулю: $\bar{\lambda_i} = |\lambda_i|$. Собирайте новую матрицу $\bar{H} = V\bar{\Lambda}V^T$ и делайте шаг Ньютона по ней.

Сравните траектории обоих подходов на невыпуклой 2D-поверхности из различных стартовых точек (особенно вблизи седловых точек). Постройте график изменения γ_k в зависимости от номера итерации. Покажите геометрически, почему метод сдвига $(H + \gamma I)$ при больших γ вырождается в обычный градиентный спуск, а спектральное усечение сохраняет "ньютоновское"поведение вдоль осей положительной кривизны. Сравните эти методы на ML-датасетах по времени одной итерации (Холецкий + подбор γ против полного спектрального разложения $O(n^3)$).

5.3.2 Трек 2. Немонотонная оптимизация: Метод Барзилая-Борвейна

Линейный поиск (Армихо/Вульф) требует дополнительных вызовов оракула функции, что замедляет время итерации Градиентного спуска. Метод

Барзилая-Борвейна - это ГС, который вычисляет длину шага α_k аналитически, аппроксимируя матрицу Гессе скаляром $\frac{1}{\alpha_k}I$

Задание:

1. Реализуйте ГС с шагом ББ. Обозначая $s_k = x_k - x_{k-1}$ и $y_k = \nabla f(x_k) - \nabla f(x_{k-1})$, длину шага можно вычислить как $\alpha_k = \frac{<s_k, s_k>}{<s_k, y_k>}$ (или $\frac{<s_k, y_k>}{<y_k, y_k>}$)

2. этот шаг **не гарантирует** убывания функции на каждой итерации, поэтому применять линейный поиск здесь не нужно.

3. Запустите алгоритм на квадратичной функции с большим числом обусловленности κ и на ML-оракулах.

Сравните траекторию обычного ГС с линейным поиском и ГС с шагом ББ на графике. Постройте и сравните кривые сходимости обычного ГС с линейным поиском и ГС с шагом ББ. Сравните время работы алгоритмов на высокоразмерных данных. Почему метод, игнорирующий монотонное убывание функции, на практике сходится гораздо быстрее классического градиентного спуска?

5.3.3 Трек 3. Диагональное предобуславливание

Сложность итерации ГС составляет $O(n)$, но он страдает от плохой обусловленности. Метод Ньютона решает проблему обусловленности за счет полного учета кривизны, но требует $O(n^3)$ для разложения Холецкого. Компромиссный вариант - учитывать только диагональные элементы матрицы Гессе (кривизну вдоль осей координат, игнорируя смешанные производные).

Задание:

1. Реализуйте модифицированный Градиентный спуск: $x_{k+1} = x_k - \alpha_k \left[Diag(\nabla^2 f(x_k))\right]^{-1} \nabla f(x_k)$. Обращение такой диагональной матрицы стоит $O(n)$.

2. Протестируйте этот метод на ваших ML-оракулах, используя линейный поиск Армихо для подбора скаляра α_k

Сравните этот алгоритм с обычным ГС и полным Ньютоном. Постройте графики «Относительная невязка от Времени работы в секундах» на «широком» датасете ($n \gg m$). Позволяет ли учет только диагональной кривизны существенно сократить число итераций ГС? Насколько этот метод быстрее полного Ньютона по времени одной итерации?

5.3.4 Трек 4. Спектральный анализ Гессиана

Поведение градиентных методов полностью определяется спектром (собственными значениями) матрицы вторых производных. Метод Ньютона, напротив, искривляет пространство так, чтобы «сплюснуть» весь спектр.

Задание:

1. На каждой 5-й итерации алгоритма Ньютона (до момента сходимости) вычисляйте полный спектр матрицы Гессе $\nabla^2 f(x_k)$ с помощью scipy.linalg.eigh или eigvalsh

2. Исследуйте распределение собственных значений. Постройте гистограмму распределения спектра в начальной точке $x_0 = 0$ и в точке оптимума x^*.

Исследуйте, как L_2-регуляризация λ влияет на распределение спектра. Покажите на графиках, присутствует ли в задаче «спектральная щель» - отрыв нулевых собственных значений от ненулевых. Опишите эволюцию числа обусловленности $\kappa = \lambda_{\max}/\lambda_{\min}$ в процессе оптимизации целевой функции.

5.3.5 Трек 5. Метод доверительных областей через Ломаную Пауэлла

Метод Ньютона с линейным поиском может совершать плохие шаги, если гессиан близок к сингулярному. Альтернативный вариант - ограничить длину шага радиусом Δ (Trust Region). Метод ломаной позволяет находить шаг в доверительной области чисто геометрически, решая обычное квадратное уравнение, без применения теории условной оптимизации.

Задание:

1. На каждой итерации вычислите шаг градиентного спуска до минимума вдоль антиградиента (точка Коши: $p^C = \frac{||g||^2}{g^T H g} g$) и полный шаг Ньютона ($p^N = -H^{-1} g$). Аппроксимируйте траекторию метода ломаной линией: от 0 к p^C, а затем от p^C к p^N.

2. Если $||p^N|| \leq \Delta$, делаем шаг Ньютона $p = p^N$

3. Если $||p^C|| \geq \Delta$, делаем укороченный градиентный шаг $p = \frac{\Delta}{||p^C||} p^C$

4. Иначе ищем точку на отрезке от p^C до p^N, которая пересекает границу сферы. Нужно найти скаляр $\tau \in [0, 1]$ такой, что $||p^C + \tau(p^N - p^C)||^2 = \Delta^2$ (Это обычное квадратное уравнение относительно τ).

5. Оцените успешность шага: вычислите отношение фактического убывания функции к предсказанному $\rho = \frac{f(x_k) - f(x_k + p)}{m_k(0) - m_k(p)}$, где $m_k(p) = f(x_k) + \langle \nabla f(x_k), p \rangle + \frac{1}{2} \left\langle \nabla^2 f(x_k) p, p \right\rangle$ - квадратичная модель функции в точке x_k. Обратите внимание, что знаменатель $m_k(0) - m_k(p)$ всегда положителен для направления спуска. При неудачном шаге ($\rho < 0.25$) радиус Δ уменьшается; при удачном ($\rho > 0.75$) - увеличивается.

Оцените вычислительную сложность такого подхода. Сравните траектории Ньютона с линейным поиском и метода ломаной на невыпуклой 2D-функции. Покажите, как метод ломаной плавно интерполирует между градиентом и Ньютоном.

Сравните эти два метода на реальных ML-оракулах из вашего варианта. Постройте кривые сходимости (относительная невязка от числа итераций) и графики производительности (невязка от времени работы в секундах). Дает ли отказ от тяжелого бэктрекинга в пользу метода доверительных областей выигрыш по времени на больших датасетах?

5.3.6 Трек 6. Прецизионная точность (float32 vs float64) и пределы алгоритмов

В машинном обучении ради скорости вычислений повсеместно используются типы данных пониженной точности (одинарная точность float32), в то время как классическая оптимизация строится на двойной точности (float64). В этом треке вы исследуете, как точность данных влияет на сходимость и вычислительный профиль алгоритмов.

Задание:

1. Принудительно приведите массивы признаков A и ответов b в тип np.float32. Запустите ГС и Метод Ньютона. Заставьте алгоритмы работать до достижения минимально возможной нормы градиента (когда алгоритм начнет стагнировать). Повторите эксперимент в np.float64

2. Настройте профилирование времени выполнения каждой итерации, разбивая его на три базовых компонента: а) вычисление градиента/гессиана (оракул), б) линейный поиск, в) решение линейной системы (разложение Холецкого). Профилирование проведите как для float32, так и для float64.

Постройте график убывания нормы градиента (в логарифмической шкале) от номера итерации для обеих точностей. Найдите момент, когда ошибка округления начинает доминировать над шагом метода. Сделайте вывод о том, какой максимальной точности оптимизации способен обеспечить метод Ньютона в float32 по сравнению с float64.

Постройте столбчатые диаграммы (или графики с областями), показывающие % времени, затрачиваемый на каждый из трех компонентов внутри одной итерации, и сравните их для float32 и float64.

Наблюдаете ли вы реальный выигрыш во времени при переходе на float32? (проанализируйте, где именно numpy/scipy ускоряются, а где время остается одинаковым, и объясните это, вспомнив как работают LAPACK/BLAS библиотеки и CPU cache).